4-bit array multiplier: RTL to GDSII

Jal Parikh · 2023102066 | Garima Mittal · 2023102069
ECE463 Digital VLSI | 25 October 2026
Repository: https://github.com/Xponent123/dvd_assgn

We implemented a registered unsigned 4 × 4 array multiplier using the row connections in RTL_to_GDS.pdf. We checked all 256 input pairs, synthesized the RTL and completed place and route in SKY130A. Our final routed design passes setup and hold analysis at a 6.5 ns clock period, with zero reported DRC and LVS errors.

Table 1. Main results. Cell area is distinct from the 22,500 µm² die area.

Key result	Value
Standalone synthesis	63 cells; 740.7104 µm²; 54.05% flip-flop area
Timing estimates	Part C: 384.025 MHz; post-route worst corner: 155.891 MHz
Configured physical operating point	153.846 MHz; 13 ns under the guide's 2T convention

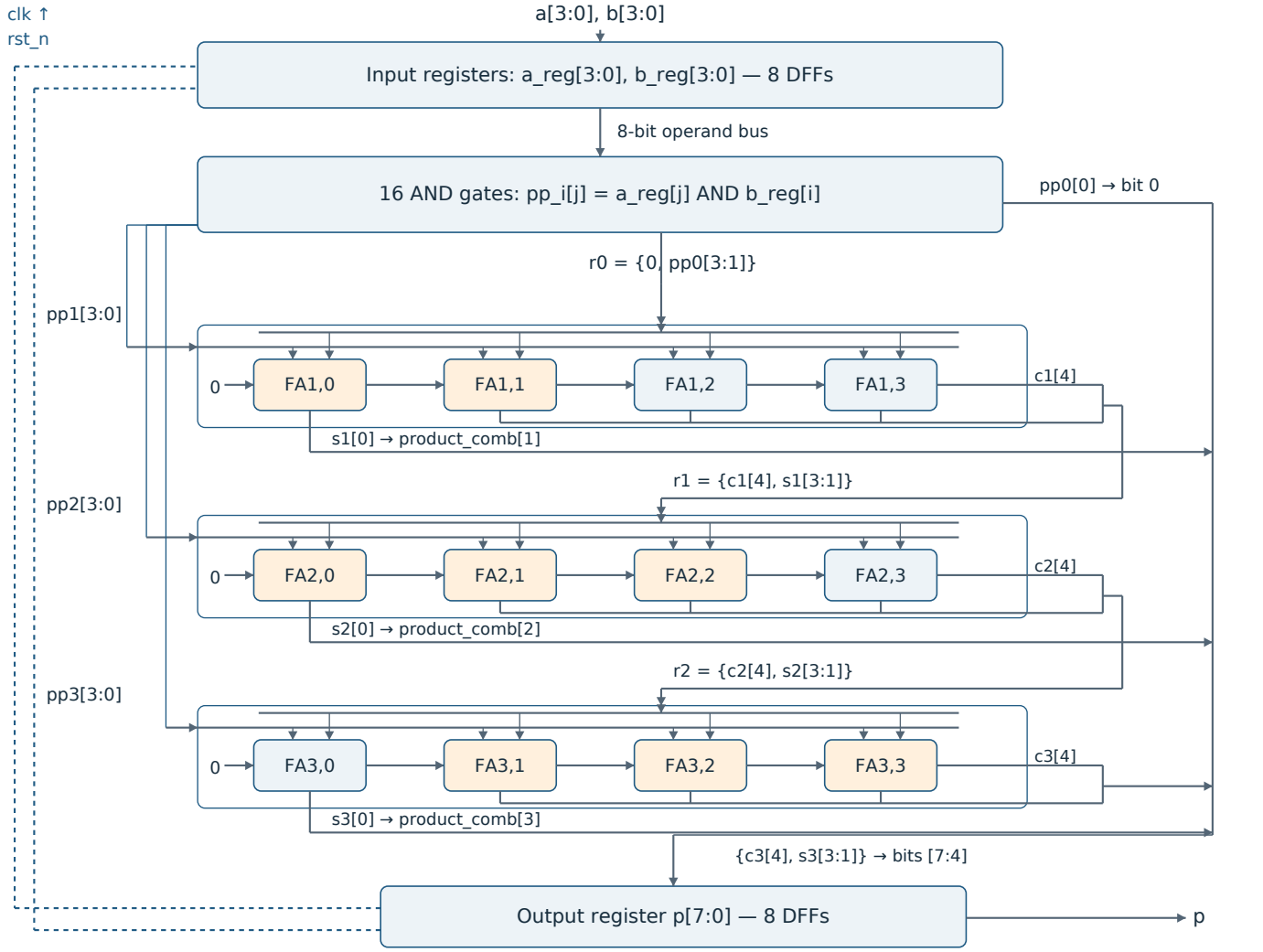


Figure 1. Connected array architecture: 16 AND operations, 12 explicit full adders and 16 register bits. Each row's two 4-bit buses feed all four adders bit by bit; horizontal arrows carry c[j] to the next adder. Pale orange marks an eight-adder structural route, not eight intact mapped timing cells. Dashed controls reach both register groups.

We used assign statements for sum = a XOR b XOR cin and cout = ab + a·cin + b·cin. Each row starts with c[0] = 0. We register the inputs at edge 1 and the result at edge 2. Latency is 2 clock cycles (inputs are captured at edge 1, the product at edge 2); following the assignment, latency in ns = 2 × T. The pipeline accepts one input pair per cycle. We used no multiplication operator in the design RTL.

Part A — verification and worked example

We generated a 10 ns clock and held rst_n low for the first 20 ns. We applied all 256 operand pairs at falling edges, waited for two rising edges, and compared p with a × b after a further 2 ns. We used case inequality so unknown values fail, and counted the comparisons instead of printing a fixed count.

```
RTL:       PASS: 256 vectors checked, 0 errors
Fault:     FAIL: 256 vectors checked, 95 errors
Restored:  PASS: 256 vectors checked, 0 errors
Mapped:    PASS: 256 vectors checked, 0 errors
```

To check the testbench, we removed cin from the sum expression in a temporary copy of full_adder.v. This produced 95 mismatches and a failing exit status. We then ran the correct RTL again and obtained PASS.

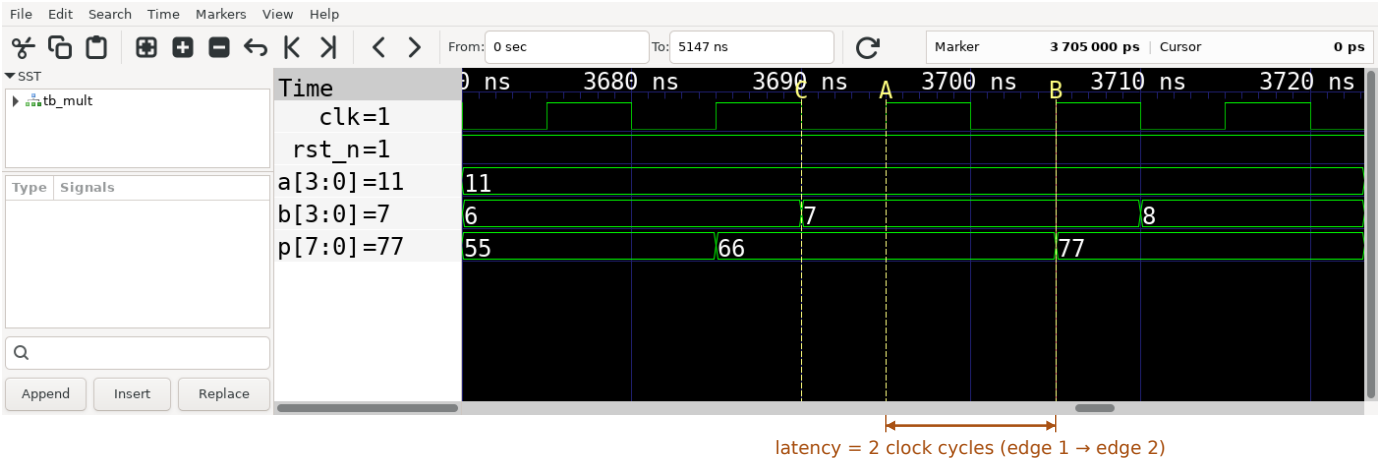


Figure 2. Our GTKWave example (decimal buses). At C (3690 ns) we apply a = 11 and b = 7. Rising edge 1 at A (3695 ns) captures the inputs into a_reg/b_reg while p still holds 66; rising edge 2 at B (3705 ns) captures p = 77. Latency is therefore 2 clock cycles.

Worked example — Jal Parikh, 2023102066

We used Jal's roll number, 2023102066. Its last digit is 6, so A = 6 + 5 = 11 = 1011_2 and B = 7 = 0111_2. The shifted partial products give 11 + 22 + 44 + 0 = 77 = 01001101_2. Our manual working and row values are shown below.

```
        1 0 1 1       A = 11
      x 0 1 1 1       B = 7
---------------
        1 0 1 1       pp0 = A AND b0 (b0 = 1)
      1 0 1 1         pp1 = A AND b1 (b1 = 1), shifted 1
    1 0 1 1           pp2 = A AND b2 (b2 = 1), shifted 2
  0 0 0 0             pp3 = A AND b3 (b3 = 0), shifted 3
---------------
0 1 0 0 1 1 0 1       P = 77
```

Table 2. Array trace for the worked example; vectors are written MSB first.

Row	pp_i = A AND b_i	Row input	s_i / carry out	Finished output
0	1011	—	—	p[0]=1; r0=0101
1	1011	0101	0000 / 1	p[1]=0; r1=1000
2	1011	1000	0011 / 1	p[2]=1; r2=1001
3	0000	1001	1001 / 0	p[3]=1; p[7:4]=0100

Additional checks. We also ran +EXTENDED in tb_mult.v. It checks consecutive-cycle inputs against a shifted-addition reference, drains and restarts the pipeline, and tests three off-edge resets. Across these tests, every register bit is first high and then checked cleared. RTL passes at 10 ns; the extended gate-level test passes at 20 ns with unit-delay models. The required gate-level test passes at 10 ns. The extra checks detect synchronous-reset and input-register-bypass faults.

Part B — logic synthesis

We ran Yosys 0.69 with sky130_fd_sc_hd__tt_025C_1v80.lib, the typical library at 25 °C and 1.80 V. We followed the seven synthesis steps in the assignment and simulated the resulting netlist with the same exhaustive testbench. A clean rerun produced identical netlist and statistics files.

Table 3. Synthesis sequence; synth.tcl invokes each operation through yosys.

Step	Operation
1	read_verilog rtl/full_adder.v rtl/mult_array.v
2	synth -top mult_array -flatten
3	dfflibmap -liberty $lib
4	abc -liberty $lib
5	opt_clean -purge
6	tee -o synth/mult_array_stat.rpt stat -liberty $lib
7	write_verilog -noattr synth/mult_array_netlist.v

Table 4. Part B results from mult_array_stat.rpt and the required 10 ns gate-level test.

Required Part B metric	Result
Total cells	63
Flip-flops	16 × dfrtp_1
Total cell area	740.7104 μm²
Flip-flop area	16 × 25.024 = 400.384 μm²
Flip-flop area fraction	54.05%
Three most-used types	dfrtp_1: 16; nand2_1: 8; a21oi_1: 6 (logic only: nand2_1: 8; a21oi_1: 6; xnor2_1: 4)
Mapped simulation result	PASS: 256 vectors checked, 0 errors

Table 5. Mapped logic breakdown; family areas are rounded independently.

Combinational family	Count	Area (μm²)
XOR / XNOR	10	130.1
NAND	13	57.6
AOI / OAI	10	60.1
AND	4	27.5
Majority	3	30.0
NOR	3	15.0
Inverters	2	7.5
Isolation-function gates	2	12.5
Total logic	47	340.3

We kept all 12 full-adder instances in RTL. Four have one input tied to zero, so synthesis can simplify them to half-adder functions. Shared terms and inverted logic also change the gate count. Our netlist uses two lpflow_isobufsrc_1 cells for A AND NOT SLEEP as ordinary logic. LibreLane excludes this family and synthesizes the RTL separately; its 68-cell result is therefore a different mapping.

Part C — static timing

We analyzed our Part B netlist in OpenSTA using the typical library, an ideal clock, 1 ns input delays on all inputs except clk, and 1 ns output delays. We first used T = 10 ns, calculated the minimum period from slack, and reran at the rounded value. The setup-critical path is in clock group clk; the reset-recovery path appears separately.

Table 6. Part C results. Estimates use three-decimal printed slack, not unrounded internal precision.

Required Part C metric	Result
Worst slack at T = 10 ns	+7.396 ns
T_min = 10 − 7.396	2.604 ns
F_max = 1000 / 2.604	384.025 MHz
Slack at T = T_min rounded to 0.01 ns (2.60 ns)	−0.004 ns (≈ 0; see note)
Additional check at T = 2.61 ns	+0.006 ns
Critical path start point	a_reg[1] (_095_)
Critical path end point	p[6] register (_092_)
Number of cells on the critical path	8 combinational; 10 including both flip-flops
Latency = 2 × T_min	5.208 ns
Throughput	384.025 million results/s

Table 7. Critical-path cells in order. All types have the sky130_fd_sc_hd__ prefix.

Instance	Cell suffix	Delay (ns)	Output / interpretation	RTL full adder
_095_/Q	dfrtp_1	0.398	Launch a_reg[1]	—
_039_/Y	clkinv_1	0.074	~a_reg[1]	— (operand inverter)
_047_/Y	o311ai_0	0.348	Factored row-1 / partial-product term	FA1,0 / FA1,1 (merged)
_050_/Y	a21oi_1	0.172	c2[1]	FA2,0 carry
_063_/X	maj3_1	0.381	c2[2]	FA2,1 carry
_064_/Y	xnor2_1	0.160	~s2[2]	FA2,2 sum
_066_/X	maj3_1	0.385	~c3[2]	FA3,1 carry
_075_/X	maj3_1	0.377	~c3[3]	FA3,2 carry
_079_/X	xnor3_1	0.175	s3[3] = product_comb[6]	FA3,3 sum
_092_/D	dfrtp_1	—	Capture p[6]; arrival 2.471 ns	—

Five cells are clean full-adder outputs (three maj3 carries, xnor2 and xnor3 sums), a21oi_1 is the FA2,0 carry, o311ai_0 merges FA1,0/FA1,1 logic, and clkinv_1 is an operand inverter. Our RTL route FA1,0 → FA1,1 → FA2,0 → FA2,1 → FA2,2 → FA3,1 → FA3,2 → FA3,3 crosses eight adders, matching the guide's 2N = 8 estimate. The match of eight mapped cells with 2N = 8 is partly a coincidence, because technology mapping merges adder stages. The path delay is 2.471 ns = 0.398 ns clk→Q + 2.073 ns of logic (about 0.26 ns per stage), plus 0.134 ns setup.

At T = 2.60 ns OpenSTA prints arrival 2.471 ns, required 2.466 ns and slack −0.004 ns, so the submitted sta/mult_array_timing.rpt is labelled VIOLATED. This is only the rounding of T_min = 2.604 ns to 2.60 ns; at 2.61 ns the slack is +0.006 ns. Fmax is a slack-derived estimate under these constraints, not a hardware measurement.

Part D — implementation and timing closure

We started LibreLane at 0.5 × ceil(1.2 × 2.604 / 0.5) = 3.5 ns, as required. We increased the period in 0.5 ns steps until setup timing passed, then made two repair attempts at 6.5 ns to clear slew violations. We used a fixed 150 × 150 µm die and retained all signoff checks.

Table 8. Final physical results. Physical-instance count includes support cells.

Required Part D metric	Final result
CLOCK_PERIOD used	6.5 ns
Number of runs needed	9: seven period trials and two repair attempts
Core area	17759.5 μm²
Cells after synthesis (Part B)	63
Cells after place and route	1901 physical instances (335 standard cells excluding fill/decap)
Kinds of cells added	Port/fanout and clock buffers, taps, fillers, decaps
Wirelength	2405 μm
Setup slack	+0.085280 ns (hold +0.124205 ns)
Post-route Fmax	155.891 MHz
Fmax from Part C	384.025 MHz
DRC errors	0 Magic, 0 KLayout
LVS errors	0

Table 9. Archived attempts. Repair = RUN_POST_GRT_DESIGN_REPAIR; Margin = GRT_DESIGN_REPAIR_MAX_SLEW_PCT. DRC M/K / LVS gives Magic DRC / KLayout DRC / LVS error counts; Slew is the slew-violation count (reset distribution).

Run	T (ns)	Setup (ns)	Repair	Margin (%)	DRC M/K / LVS	Slew	Result
1	3.5	-2.129189	No	10	0/0 / 0	0	FAIL (setup)
2	4.0	-1.629189	No	10	0/0 / 0	0	FAIL (setup)
3	4.5	-1.139431	No	10	0/0 / 0	0	FAIL (setup)
4	5.0	-0.641733	No	10	0/0 / 0	0	FAIL (setup)
5	5.5	-0.141733	No	10	0/0 / 0	0	FAIL (setup)
6	6.0	-0.064138	No	10	0/0 / 0	11	FAIL (setup, slew)
7	6.5	+0.085280	No	10	0/0 / 0	11	FAIL (slew)
8	6.5	+0.085280	Yes	10	0/0 / 0	11	FAIL (slew)
9	6.5	+0.085280	Yes	20	0/0 / 0	0	PASS

In runs 8 and 9, we enabled RUN_POST_GRT_DESIGN_REPAIR. The 10% repair margin left 11 slew violations. We increased GRT_DESIGN_REPAIR_MAX_SLEW_PCT to 20, after which the final count was zero. We kept the 0.750 ns transition limit. The table counts nine implementation attempts; a later verification rerun is separate.

We took the final values from timing__setup__ws, timing__hold__ws, route__wirelength, design__core__area, design__instance__count, magic__drc_error__count, klayout__drc_error__count and design__lvs_error__count in metrics.json. We used final signoff values rather than intermediate routing estimates.

Physical results — interpretation

Table 10. Reconciliation from statistics, metrics and final netlist/DEF. The two synthesis results are separate mappings.

Stage / role	Instances	Evidence
Standalone Part B synthesis	63	Part B Yosys statistics
LibreLane synthesis	68	LibreLane synthesis report
Input/output port buffers	17	9 input* and 8 output* instances
Reset fanout buffers	2	fanout18: buf_2; fanout19: clkdlybuf4s50_1
Clock buffers	3	clkbuf_16
Functional cells plus buffers	90	68 + 17 + 2 + 3
Tap cells	245	Well/substrate bias
Stdcell-count metric	335	90 + 245; fill class excluded
Fillers / decaps	269 / 1297	Row continuity / local supply support
Total physical instances	1901	335 + 269 + 1297

We checked the repair logs to identify the added buffers. Post-placement repair inserted 17 port buffers and two reset-fanout buffers. Both fanout buffers initially used delay cells. Post-global-route repair changed fanout18 to buf_2; fanout19 remained clkdlybuf4s50_1. CTS added three clkbuf_16 cells.

Our fill/decap area is 16,523.3 μm², about 93.0% of the core. The reported stdcell area occupies about 6.96%; the 68 functional cells alone occupy about 4.25%. Most instances are therefore support cells. Taps provide well/substrate bias, fillers maintain row continuity, and decaps provide local supply capacitance.

Table 11. Operating point versus estimates; the final setup corner is max_ss_100C_1v60.

Timing quantity	Value / condition
Calculated rate at configured period	1000 / 6.5 = 153.846 MHz; 153.846 million results/s
Guide latency at configured period	2 × 6.5 = 13 ns
Worst-corner estimate	1000 / (6.5 − 0.085280) = 155.891 MHz
Nominal-RC, typical estimate	1000 / (6.5 − 3.292673) = 311.786 MHz
Final worst critical path	a_reg[0] (_104_) → p[6] (_102_)
Final constraint context	0.25 ns uncertainty; 1.3 ns I/O delays; 0.95/1.05 derates

For Part C, we used typical cells and an ideal clock. Final signoff includes extracted RC and propagated clocks, and its worst setup corner is slow process, 100 °C, 1.60 V. The netlist and critical path also differ. We therefore cannot attribute the frequency change to wires alone. The I/O delays are listed for context; they do not add directly to this register-to-register path.

Table 12. Setup and hold slack per corner at 6.5 ns.

Corner	Setup WS (ns)	Hold WS (ns)
nom_tt_025C_1v80	3.293	0.350
nom_ss_100C_1v60	0.125	0.363
nom_ff_n40C_1v95	4.320	0.130
min_tt_025C_1v80	3.319	0.342
min_ss_100C_1v60	0.160	0.362
min_ff_n40C_1v95	4.329	0.124
max_tt_025C_1v80	3.269	0.357
max_ss_100C_1v60	0.085	0.362
max_ff_n40C_1v95	4.314	0.134

Fmax falls from 384.025 MHz (Part C: typical, ideal clock, no wires) to 311.786 MHz (routed, nominal typical corner) and 155.891 MHz (routed, worst corner max_ss_100C_1v60). The slow corner, not the added cells, sets the final period.

Total power is 0.353 mW. The flow inserted no hold-repair buffers and no antenna diodes.

Physical design — annotated layout views

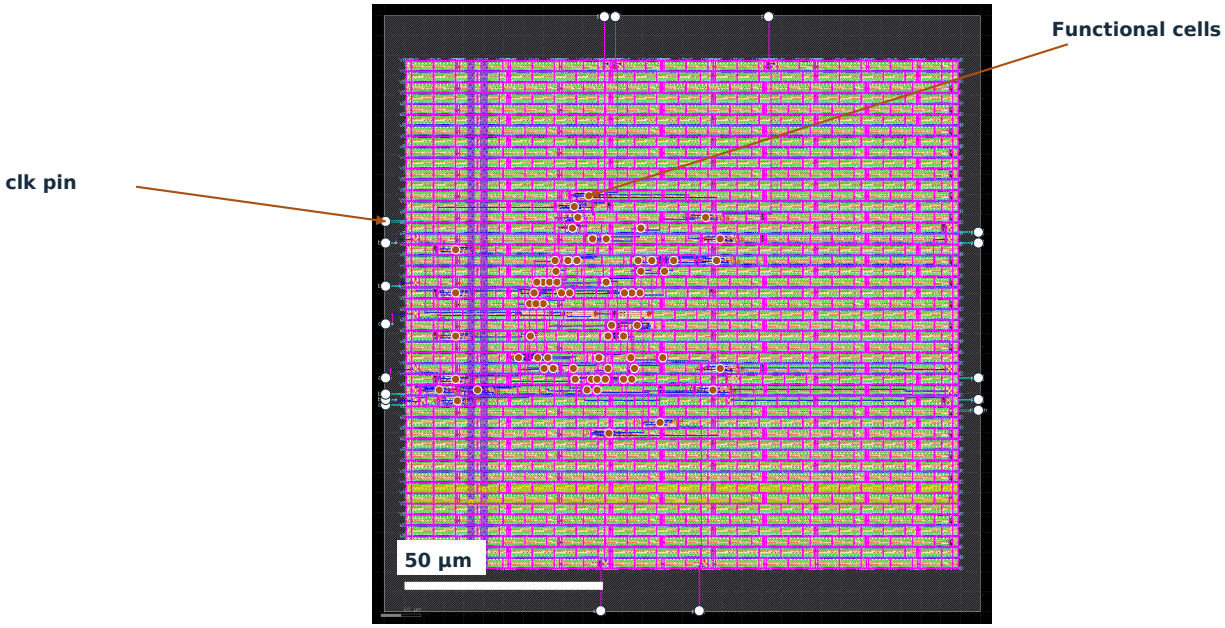


Figure 3. Our final 150 × 150 µm die in KLayout. We marked functional cells with orange dots and signal pins with open circles using their layout coordinates. The large area around the logic is mainly occupied by physical support cells.



Figure 4. Our zoomed KLayout view, x = 40–95 µm and y = 42.8125–77.1875 µm. We used GDS instance positions for the cell labels and final DEF coordinates for the VGND rail and signal _000_. The scale bars use layout distances.

li1 met1 met2 met3 met4 met5

We used the SKY130 KLayout palette for li1 and met1–met5. The colors identify layers; the cell labels identify their functions.

Answers to the six report questions

1. Logic-cell count and mapping

Synthesis produced 63 − 16 = 47 logic cells, compared with 16 + 12 = 28 RTL blocks. The cell library has no single full-adder block for ABC to use here, so each full adder becomes separate sum and carry cells: we have 10 XOR/XNOR and 3 maj3 cells. The 16 AND gates are partly absorbed into NAND/AOI cells (13 NAND-family cells), four adders with a constant-0 input simplify to half-adder logic, and shared terms are factored across adder boundaries. XOR/XNOR cells occupy about 38% of the combinational area. We kept all 12 adders in RTL.

2. Critical path and the 2N estimate

Our Part C critical path runs from register a_reg[1] (_095_/Q) to register p[6] (_092_/D) through eight combinational cells (ten with both flip-flops). Its arrival is 2.471 ns: 0.398 ns clk→Q plus 2.073 ns of logic, about 0.26 ns per cell. Five cells are clean full-adder outputs, a21oi_1 is the FA2,0 carry, o311ai_0 merges FA1,0/FA1,1 logic and clkinv_1 inverts an operand bit. The RTL route crosses eight adders, consistent with 2N = 8; the equal count of mapped cells is partly a coincidence of technology mapping.

3. Flip-flop share of cell area

We calculated 16 × 25.024 / 740.7104 × 100 = 54.05%. Each resettable dfrtp_1 (25.024 µm²) holds a master–slave latch pair plus reset logic and is about 3.5× the average combinational cell (340.3264 / 47 = 7.24 µm²). The register count is fixed by the I/O widths (8 input bits, 8 output bits), while the 4-bit array needs only 47 small gates, so flip-flop area (400.4 µm²) exceeds all logic area (340.3 µm²). For larger N the share falls, because logic grows about as N^2 and registers only as 4N.

4. Post-route frequency and added cells

Post-route Fmax is 155.891 MHz versus 384.025 MHz in Part C. Part C used a typical corner, an ideal clock and no wires; signoff adds extracted wire RC, a propagated clock tree, 0.25 ns uncertainty and 0.95/1.05 derates, and its worst corner is slow (ss, 100 °C, 1.60 V). At the nominal typical corner the same layout gives 311.786 MHz. The flow added 17 port buffers (drive and isolate the I/O pins), 2 reset-fanout buffers (fix rst_n slew and fanout), 3 clkbuf_16 clock buffers (distribute the clock with low skew), 245 tap cells (tie wells and substrate to the rails to prevent latch-up), 269 fillers (keep wells, implants and power rails continuous) and 1297 decaps (local supply decoupling). Taps, fillers and decaps are off the signal paths and add no delay.

5. Why both DRC and LVS must pass

DRC checks layout geometry against the foundry's manufacturing rules (width, spacing, enclosure, density, antenna), but it knows nothing about the intended circuit. LVS extracts devices and nets from the layout and compares them with the netlist, catching opens, shorts and missing or extra devices, but it does not check manufacturability. A layout can pass one and fail the other, and masks are expensive and cannot be corrected after fabrication, so both must pass. We obtained zero Magic DRC, zero KLayout DRC and zero LVS errors; the router's own DRC count went 9 → 3 → 4 → 0 over its iterations.

6. Scaling from four to eight bits

Using the guide's formulas, AND gates grow from $4^2 = 16$ to $8^2 = 64$ (4×), full adders from $4(4-1) = 12$ to $8(8-1) = 56$ (4.67×), the critical path from $2\times4 = 8$ to $2\times8 = 16$ full-adder delays, and flip-flops from 16 to 32. Using our own numbers, the register overhead is 0.398 ns clk→Q + 0.134 ns setup = 0.532 ns, and the logic costs 2.073 / 8 ≈ 0.259 ns per stage. An 8-bit T_min is therefore about $0.532 + 16 \times 0.259 \approx 4.68$ ns, so Fmax ≈ 214 MHz (vs 384 MHz) and latency ≈ 9.4 ns. Simply doubling T_min (5.21 ns) would overestimate it, because the register overhead does not scale. This is an estimate; we have not synthesized an 8-bit version.

Conclusion and assumptions

We verified our registered 4-bit array multiplier at RTL and gate level for all 256 input pairs. The deliberate sum fault produced 95 errors, so our testbench detects the required fault. We also checked consecutive-cycle operation and off-edge reset. Our final routed design meets setup and hold timing at 6.5 ns with zero reported DRC and LVS errors.

The mapped design uses 63 cells, with 54.05% of cell area in the 16 flip-flops. At the configured 6.5 ns period, the calculated clock rate and throughput are 153.846 MHz and 153.846 million results/s. We report 13 ns latency using the assignment's 2T convention. The slack-derived post-route Fmax is 155.891 MHz.

Individual contributions

Both members contributed equally to this assignment. Jal Parikh (2023102066) wrote the RTL (full_adder.v, mult_array.v), the STA script with the T_min/F_max analysis, and the LibreLane configuration and timing-closure runs. Garima Mittal (2023102069) wrote the exhaustive self-checking testbench and the fault-injection test, the synthesis script with gate-level simulation, and the physical-design analysis (metrics, DRC/LVS checks and layout views). We debugged the design, answered the report questions and reviewed the README and report together.

Assumptions and limits

Latency and throughput. Latency is 2 clock cycles (inputs are captured at edge 1, the product at edge 2); following the assignment, latency in ns = 2 × T. Our throughput calculations assume a steady stream of inputs, one pair per clock, after the pipeline fills and with reset inactive.

Timing estimates. We calculate Fmax from T minus reported slack. This assumes the same path delays, netlist and constraints when changing the clock requirement; a new implementation can change them. Part C uses an ideal clock and typical cells, while final signoff uses extracted RC and multiple corners. These are tool-analysis results; we have not measured fabricated hardware.

Eight-bit estimate. The Q6 estimate (about 4.68 ns and 214 MHz) assumes similar delay per adder stage and unchanged register overhead. Mapping, fanout and wire capacitance can change the total delay, so it is not a measured result.

Additional verification. The gate-level models use artificial unit delays and are a functional check. Our optional formal proof starts from zero initialization and uses async2sync; we checked off-edge asynchronous reset separately in simulation.

Saved results. Our GDS and metrics come from attempt_09_6p5ns_repair. We rechecked front-end results and archived physical reports for this write-up; we did not rerun place and route for these documentation changes. README.md contains our commands, all nine attempt settings, and the steps that copy metrics.json and the GDS from runs/attempt_09_6p5ns_repair/final/ into pnr/.

Appendix — our verification results

Simulation summaries

rtl.log

```
PASS: 256 vectors checked, 0 errors
```

gls.log

```
PASS: 256 vectors checked, 0 errors
```

rtl_extended.log

```
PASS: 256 vectors checked, 0 errors
```

EXTENDED PASS: 256 unique pairs streamed, exact pipeline latency checked, 3 off-edge resets, all 16 register bits exercised

gls_extended.log

```
PASS: 256 vectors checked, 0 errors
```

EXTENDED PASS: 256 unique pairs streamed, exact pipeline latency checked, 3 off-edge resets, all 16 register bits exercised

Deliberate fault and regression checks

```
MISMATCH: a=3 b=3 expected=9 got=5 time=1067000
MISMATCH: a=3 b=6 expected=18 got=10 time=1127000
MISMATCH: a=3 b=7 expected=21 got=9 time=1147000
MISMATCH: a=3 b=11 expected=33 got=29 time=1227000
MISMATCH: a=3 b=12 expected=36 got=20 time=1247000
FAIL: 256 vectors checked, 95 errors
```

Table A1. Checks run in isolated copies. Expected mutant failures are successful test outcomes.

Check	Observed result
Incomplete exhaustive coverage	240 comparisons → failure, despite zero arithmetic mismatches
Synchronous-reset mutant	ASYNC RESET failure before the next clock edge
Input-register-bypass mutant	PIPELINE failure: previous transaction expected
Reordered initial blocks	Reset still released at 20 ns
Temporal induction	Induction step proven: SUCCESS!

STA and final signoff

```
T = 10.00 ns: worst slack max  7.396
T =  2.60 ns: worst slack max -0.004
T =  2.61 ns: worst slack max  0.006
```

We checked final setup slack +0.085280 ns and hold slack +0.124205 ns, with zero Magic/KLayout DRC, LVS, slew and antenna violations. Netgen reports “Netlists match uniquely.” We checked the final supply connectivity after the earlier VGND/VPWR floating-net warnings.

We used these SHA-256 values to check that the submitted GDS and metrics match the final run:

```
GDS:
05b512ab995d81d6ea0dcfce9f6392aa85c789697f64fbed7ec1492e7da7763f
metrics.json:
a8a93aabb3ffeb669cf32eeecd9e277e823be3bb8c31bd62a1c028b69c4747e1
```